

Driver ECU ↔ Door ECU 에서의 CAN 사이 - Bluetooth기반 교통약자용 버스 전동문 및 전동경사로 소규모 통합 제어 시스템

```
1  # Driver ECU ↔ Door ECU CAN 사용 문서
2
3  ## 1. 문서 목적
4
5  이 문서는 **CAN 라이브러리 자체 설명**이 아니라,
6  **운전석 ECU 개발자**와 **door ECU 개발자**가 실제로 바로 써야 하는 내용만 정리
7
8  즉, 아래 두 가지에만 집중한다.
9
10 - 어떤 프레임을 주고받는가
11 - 제공된 래퍼 API를 어떤 순서로 쓰면 되는가
12
13 CAN 라이브러리 내부 계층(`CANSocket`, `CANCore`, `CANPlatform`, binding,
14 이 문서에서는 실제 사용에 필요한 범위만 간단히 언급한다.
15
16 ---
17
18 ## 2. 개발자가 실제로 사용하는 파일
19
20 ### 운전석 ECU 측
21
22 - `driver_ecu_can.h`
23 - `driver_with_door_protocol.h`
24
25 ### door ECU 측
26
27 - `door_ecu_can.h`
28 - `driver_with_door_protocol.h`
29
30 일반적인 애플리케이션 코드는 거의 이 두 래퍼만 보면 된다.
31
32 - 운전석 ECU는 `DriverECUCAN`으로 **command 송신 / status 수신**
33 - door ECU는 `DoorECUCAN`으로 **command 수신 / status 송신**
34
35 `driver_with_door_protocol.h`는 CAN ID와 payload bit layout을 정의하는 공
36
37 ---
38
39 ## 3. 내부 계층은 어떻게 연결되는가
40
41 사용자는 보통 `driverECUCANOpen()` 또는 `doorECUCANOpen()`만 호출하면 된다.
42
43 그 아래에서는 대략 아래처럼 연결된다.
44
45 ```text
46 Application
47   └─ DriverECUCAN / DoorECUCAN   보드/MCU 자원 매핑
48   ```
49
50 여기서 사용자가 알아둘 점은 하나다.
51
52 - `config.port_name`에 예를 들어 `"can0"`를 넣으면,
53 - 내부 binding이 그 이름에 맞는 실제 CAN 포트/핀/노드를 찾아 연결한다.
54
55 즉, 애플리케이션 개발자는 보통 **`port_name`과 bitrate만 맞게 지정**하면 된다.
56
57 ---
58
59 ## 4. 통신 방향
60
61 ### 운전석 ECU → door ECU
```

```

62
63 - CAN ID: `0x100`
64 - 의미: command frame
65
66 ### door ECU → 운전석 ECU
67
68 - CAN ID: `0x101`
69 - 의미: status frame
70
71 현재 래퍼는 다음 조건을 전제로 한다.
72
73 - Standard CAN ID 사용
74 - Classical CAN 사용
75 - payload는 현재 1바이트만 사용
76
77 ---
78
79 ## 5. 프레임 구조
80
81 ## 5.1 Command Frame (`0x100`)
82
83 운전석 ECU가 door ECU로 보내는 1바이트 payload 구조:
84
85 ```text
86 bit 0~1 : door command
87 bit 2~3 : ramp(slope) command
88 bit 4 : ramp_manual
89 bit 5 : emergency_stop
90 bit 6 : reset_fault
91 bit 7 : reserved
92 ```
93
94 ### door command 값
95
96 - `0`: `DOOR_CMD_NOP`
97 - `1`: `DOOR_CMD_OPEN`
98 - `2`: `DOOR_CMD_CLOSE`
99 - `3`: `DOOR_CMD_STOP`
100
101 ### ramp(slope) command 값
102
103 - `0`: `RAMP_CMD_NOP`
104 - `1`: `RAMP_CMD_DEPLOY`
105 - `2`: `RAMP_CMD_STOW`
106 - `3`: `RAMP_CMD_STOP`
107
108 ### flag 의미
109
110 - `ramp_manual = false` : 자동 slope 명령
111 - `ramp_manual = true` : 수동 slope 명령
112 - `emergency_stop = true` : 비상 정지 요청
113 - `reset_fault = true` : fault reset 요청
114
115 ---
116
117 ## 5.2 Status Frame (`0x101`)
118
119 door ECU가 운전석 ECU로 보내는 1바이트 payload 구조:
120
121 ```text
122 bit 0 : pinch_detected
123 bit 1~2 : door_state

```

```

124 bit 3~4 : ramp_state
125 bit 5 : fault_present
126 bit 6 : alive_toggle
127 bit 7 : reserved
128 ```
129
130 ### door state 값
131
132 - `0`: `DOOR_STATE_CLOSED`
133 - `1`: `DOOR_STATE_OPENED`
134 - `2`: `DOOR_STATE_MOVING`
135 - `3`: `DOOR_STATE_FAULT`
136
137 ### ramp state 값
138
139 - `0`: `RAMP_STATE_STOWED`
140 - `1`: `RAMP_STATE_DEPLOYED`
141 - `2`: `RAMP_STATE_MOVING`
142 - `3`: `RAMP_STATE_FAULT`
143
144 ### flag 의미
145
146 - `pinch_detected = true` : 끼임 감지됨
147 - `fault_present = true` : fault 존재
148 - `alive_toggle` : status를 publish할 때마다 토글되는 비트
149
150 참고로 현재 래퍼 API의 alive 판정은 `alive_toggle` 변화 자체를 해석하는 방식이
151 **마지막 수신 시각과 timeout 비교**로 수행된다.
152
153 ---
154
155 ## 6. 운전석 ECU 개발자가 쓰는 방법
156
157 운전석 ECU는 `DriverECUCAN` 하나를 사용하면 된다.
158
159 ### 6.1 역할
160
161 - door ECU로 command 전송
162 - door ECU가 보낸 status polling
163 - 마지막 status snapshot 캐시
164 - timeout 기반 alive 판정
165
166 ### 6.2 기본 사용 순서
167
168 #### 1) 객체와 설정 초기화
169
170 ```c
171 DriverECUCAN driver;
172 DriverECUCANConfig cfg;
173
174 driverECUCANInit(&driver);
175 driverECUCANInitConfig(&cfg);
176 ```
177
178 기본값은 다음과 같다.
179
180 ```c
181 cfg.port_name = "can0";
182 cfg.nominal_bitrate = 500000U;
183 ```
184
185 필요하면 수정:
186
187 ```c

```

```

188 cfg.port_name = "can0";
189 cfg.nominal_bitrate = 500000U;
190 ```
191
192 ##### 2) open
193
194 ```c
195 CANStatus st = driverECUCANOpen(&driver, &cfg);
196 ```
197
198 이 단계에서 내부적으로 command/status 용 CAN 소켓이 열리고,
199 `port_name`에 맞는 platform binding이 연결된다.
200
201 ##### 3) command 송신
202
203 ```c
204 st = driverECUCANSendCommand(
205     &driver,
206     DOOR_CMD_OPEN,
207     RAMP_CMD_NOP,
208     false,
209     false,
210     false
211 );
212 ```
213
214 인자 의미:
215
216 - `door_cmd`
217 - `ramp_cmd`
218 - `ramp_manual`
219 - `emergency_stop`
220 - `reset_fault`
221
222 이 함수는 내부에서 command payload를 pack해서 `0x100`으로 송신한다.
223
224 ##### 4) status polling
225
226 ```c
227 bool updated = false;
228 uint32_t now_ms = AppGetTickMs();
229
230 st = driverECUCANPollStatus(&driver, now_ms, &updated);
231 ```
232
233 동작은 다음과 같다.
234
235 - 새 status frame이 없으면: `updated = false`, 반환값은 `CAN_STATUS_OK`
236 - 새 status frame이 있으면: 내부 snapshot 갱신 후 `updated = true`
237
238 즉, 이 API는 **event callback 방식이 아니라 polling 방식**이다.
239
240 ##### 5) 최신 status 읽기
241
242 ```c
243 const DriverECUCANStatusSnapshot* s = driverECUCANGetStatus(&driver);
244 if (s != 0 && s->status_received)
245 {
246     /* s->door_state, s->ramp_state, s->pinch_detected 등 사용 */
247 }
248 ```
249
250 ##### 6) alive 확인
251
252 ```c

```

```

253 bool alive = driverECUCANIsStatusAlive(&driver, now_ms, 1000U);
254 ```
255
256 - 최근 1000ms 내 status 수신이면 `true`
257 - 아니면 `false`
258
259 ##### 7) close
260
261 ```c
262 driverECUCANClose(&driver);
263 ```
264
265 ---
266
267 ## 7. door ECU 개발자가 쓰는 방법
268
269 door ECU는 `DoorECUCAN` 하나를 사용하면 된다.
270
271 ### 7.1 역할
272
273 - 운전석 ECU가 보낸 command polling
274 - 마지막 command snapshot 캐시
275 - timeout 시 안전 fallback 적용
276 - 현재 상태를 status frame으로 송신
277
278 ### 7.2 기본 사용 순서
279
280 ##### 1) 객체와 설정 초기화
281
282 ```c
283 DoorECUCAN door;
284 DoorECUCANConfig cfg;
285
286 doorECUCANInit(&door);
287 doorECUCANInitConfig(&cfg);
288 ```
289
290 기본값:
291
292 ```c
293 cfg.port_name = "can0";
294 cfg.nominal_bitrate = 500000U;
295 ```
296
297 ##### 2) open
298
299 ```c
300 CANStatus st = doorECUCANOpen(&door, &cfg);
301 ```
302
303 이 단계에서 내부적으로 `0x100` command frame 수신용 filter가 설정된다.
304
305 ##### 3) command polling
306
307 ```c
308 bool updated = false;
309 uint32_t now_ms = AppGetTickMs();
310
311 st = doorECUCANPollCommand(&door, now_ms, &updated);
312 ```
313
314 동작:
315
316 - 새 command가 없으면: `updated = false`, 반환값은 `CAN_STATUS_OK`
317 - 새 command가 있으면: 내부 snapshot 갱신 후 `updated = true`

```

```

318
319 ##### 4) 최신 command 읽기
320
321 ```c
322 const DoorECUCANCommandSnapshot* cmd = doorECUCANGetCommand(&door);
323 if (cmd != 0 && cmd->command_received)
324 {
325     /* cmd->door_command, cmd->ramp_command, cmd->ramp_manual 등 사용 */
326 }
327 ```
328
329 ##### 5) command alive 확인
330
331 ```c
332 bool alive = doorECUCANIsCommandAlive(&door, now_ms, 1000U);
333 ```
334
335 ##### 6) timeout 시 안전 fallback 적용
336
337 ```c
338 doorECUCANForceSafeCommand(&door);
339 ```
340
341 이 함수는 현재 캐시된 command를 아래 값으로 덮어쓴다.
342
343 - `door_command = DOOR_CMD_STOP`
344 - `ramp_command = RAMP_CMD_STOP`
345 - `ramp_manual = false`
346 - `emergency_stop = false`
347 - `reset_fault = false`
348
349 즉, command timeout이나 통신 이상 시 application이 이 함수를 호출해서
350 **기본 안전 정지 상태**로 되돌릴 수 있다.
351
352 ##### 7) status publish
353
354 ```c
355 st = doorECUCANPublishStatus(
356     &door,
357     false,
358     DOOR_STATE_OPENED,
359     RAMP_STATE_STOWED,
360     false
361 );
362 ```
363
364 인자 의미:
365
366 - `pinch_detected`
367 - `door_state`
368 - `ramp_state`
369 - `fault_present`
370
371 이 함수는 내부에서:
372
373 - status cache 갱신
374 - `alive_toggle` 반전
375 - status payload pack
376 - `0x101`으로 송신
377
378 을 수행한다.
379
380 ##### 8) 현재 publish cache 확인
381
382 ```c
383 const DoorECUCANStatusCache* cache = doorECUCANGetStatusCache(&door);

```

```

384   ``
385
386   #### 9) close
387
388   ``c
389   doorECUCANClose(&door);
390   ``
391
392   ---
393
394   ## 8. 권장 루프 예시
395
396   ## 8.1 운전석 ECU 예시
397
398   ``c
399   DriverECUCAN driver;
400   DriverECUCANConfig cfg;
401   bool updated;
402   uint32_t now_ms;
403
404   driverECUCANInit(&driver);
405   driverECUCANInitConfig(&cfg);
406   driverECUCANOpen(&driver, &cfg);
407
408   for (;;)
409   {
410       now_ms = AppGetTickMs();
411
412       /* 필요 시 command 송신 */
413       driverECUCANSendCommand(
414           &driver,
415           DOOR_CMD_OPEN,
416           RAMP_CMD_NOP,
417           false,
418           false,
419           false
420       );
421
422       /* status 수신 */
423       updated = false;
424       driverECUCANPollStatus(&driver, now_ms, &updated);
425
426       if (updated)
427       {
428           const DriverECUCANStatusSnapshot* s = driverECUCANGetStatus(&driver);
429           (void)s;
430           /* UI 반영, 상태 머신 반영 등 */
431       }
432
433       if (!driverECUCANIsStatusAlive(&driver, now_ms, 1000U))
434       {
435           /* 통신 이상 처리 */
436       }
437   }
438   ``
439
440   ## 8.2 door ECU 예시
441
442   ``c
443   DoorECUCAN door;
444   DoorECUCANConfig cfg;

```

```

445 bool updated;
446 uint32_t now_ms;

447
448 doorECUCANInit(&door);
449 doorECUCANInitConfig(&cfg);
450 doorECUCANOpen(&door, &cfg);

451
452 for (;;)
453 {
454     now_ms = AppGetTickMs();

455
456     /* command 수신 */
457     updated = false;
458     doorECUCANPollCommand(&door, now_ms, &updated);

459
460     if (updated)
461     {
462         const DoorECUCANCommandSnapshot* cmd = doorECUCANGetCommand(&door,
463         (void)cmd;
464         /* door/slope 제어 로직 반영 */
465     }

466
467     if (!doorECUCANIsCommandAlive(&door, now_ms, 1000U))
468     {
469         doorECUCANForceSafeCommand(&door);
470         /* 안전 상태 반영 */
471     }

472
473     /* 현재 상태 publish */
474     doorECUCANPublishStatus(
475         &door,
476         false,
477         DOOR_STATE_MOVING,
478         RAMP_STATE_STOWED,

479         false
480     );
481 }
482 ...
483
484 ---
485
486 ## 9. 실사용 시 기억할 점
487
488 - 이 래퍼는 **polling 기반**이다.
489 - command/status 모두 현재 **1바이트 payload**만 쓴다.
490 - 운전석 ECU는 `0x100` 송신, `0x101` 수신이다.
491 - door ECU는 `0x100` 수신, `0x101` 송신이다.
492 - timeout 기반 alive 판정은 application이 주기적으로 `now_ms`를 넣어 호출해야 한다.
493 - timeout 이후 실제 하드웨어를 어떻게 안전 정지할지는 application 로직이 결정한다.
494 - `doorECUCANForceSafeCommand()`는 **수신 snapshot을 안전값으로 덮어쓰는 he
495
496 ---
497
498 ## 10. 빠른 참고
499
500 ### 운전석 ECU
501
502 1. `driverECUCANInit()`
503 2. `driverECUCANInitConfig()`
504 3. `driverECUCANOpen()`
505 4. 필요 시 `driverECUCANSendCommand()`

```

```
506 5. 주기적으로 `driverECUCANPollStatus()`
507 6. `driverECUCANGetStatus()` / `driverECUCANIsStatusAlive()` 사용
508 7. 종료 시 `driverECUCANClose()`
509
510 ### door ECU
511
512 1. `doorECUCANInit()`
513 2. `doorECUCANInitConfig()`
514 3. `doorECUCANOpen()`
515 4. 주기적으로 `doorECUCANPollCommand()`
516 5. `doorECUCANGetCommand()` / `doorECUCANIsCommandAlive()` 사용
517 6. 필요 시 `doorECUCANForceSafeCommand()`
518 7. 주기적으로 `doorECUCANPublishStatus()`
519 8. 종료 시 `doorECUCANClose()`
520
```

Driver ECU → Door ECU CAN 사용 문서

1. 문서 목적

이 문서는 CAN 라이브러리 자체 설명이 아니라, 운전석 ECU 개발자와 door ECU 개발자가 실제로 바로 써야 하는 내용만 정리한 문서이다.

즉, 아래 두 가지에만 집중한다.

- 어떤 프레임들 주고받는가
- 제공된 레퍼 API를 어떤 순서로 쓰면 되는가

CAN 라이브러리 내부 계층(CANSocket, CANCore, CANPlatform, binding, board driver)의 자세한 구조는 별도 문서를 참고하면 된다. 이 문서에서는 실제 사용에 필요한 범위만 간단히 언급한다.

2. 개발자가 실제로 사용하는 파일

운전석 ECU 쪽

- driver_ecu_can.h
- driver_with_door_protocol.h

door ECU 쪽

- door_ecu_can.h
- driver_with_door_protocol.h

일반적인 애플리케이션 코드는 거의 이 두 레퍼만 보면 된다.

- 운전석 ECU는 DriverECUCAN으로 **command** 송신 / **status** 수신
- door ECU는 DoorECUCAN으로 **command** 수신 / **status** 송신

driver_with_door_protocol.h는 CAN ID와 payload bit layout을 정의하는 공통 헤더이다.

3. 내부 계층은 어떻게 연결되는가

사용자는 보통 driverECUCANOpen() 또는 doorECUCANOpen()만 호출하면 된다.

그 아래에서는 대략 아래처럼 연결된다.

```
Application
├─ DriverECUCAN / DoorECUCAN  <- 사용자가 직접 쓰는 계층
│   └─ CANSocket              <- 공통 CAN 송수신 레퍼
│       └─ CANPlatform         <- board port와 실제 CAN 자원 연결
│           └─ binding          <- port_name -> 보드/ECU 자원 매핑
```

여기서 사용자가 알아둘 점은 하나다.

- config.port_name에 예를 들어 "can0"를 넣으면,
- 내부 binding이 그 이름에 맞는 실제 CAN 포트/핀/노드를 찾아 연결한다.

즉, 애플리케이션 개발자는 보통 port_name과 bitrate만 맞게 지정하면 된다.

4. 통신 방향

운전석 ECU → door ECU

- CAN ID: 0x100
- 의미: command frame

door ECU → 운전석 ECU

- CAN ID: 0x101
- 의미: status frame

현재 레퍼는 다음 조건을 전제로 한다.

- Standard CAN ID 사용
- Classical CAN 사용
- rawload는 현재 1비이트만 사용

5. 프레임 구조

5.1 Command Frame (8x100)

운전석 ECU가 door ECU로 보내는 1바이트 payload 구조:

```
bit 0-1 : door_command  
bit 2-3 : ramp(slope)_command  
bit 4   : ramp_manual  
bit 5   : emergency_stop  
bit 6   : reset_fault  
bit 7   : reserved
```

door_command 값

- 0: DOOR_CMD_NOP
- 1: DOOR_CMD_OPEN
- 2: DOOR_CMD_CLOSE
- 3: DOOR_CMD_STOP

ramp(slope)_command 값

- 0: RAMP_CMD_NOP
- 1: RAMP_CMD_DEPLOY
- 2: RAMP_CMD_STOP
- 3: RAMP_CMD_STOP

flag 의미

- ramp_manual = false: 자동 slope 명령
- ramp_manual = true: 수동 slope 명령
- emergency_stop = true: 비상 정지 요청
- reset_fault = true: fault reset 요청

5.2 Status Frame (8x101)

door ECU가 운전석 ECU로 보내는 1바이트 payload 구조:

```
bit 0   : pinch_detected  
bit 1-2 : door_state  
bit 3-4 : ramp_state  
bit 5   : fault_present  
bit 6   : alive_toggle  
bit 7   : reserved
```

door_state 값

- 0: DOOR_STATE_CLOSED
- 1: DOOR_STATE_OPENED
- 2: DOOR_STATE_MOVING
- 3: DOOR_STATE_FAULT

ramp_state 값

- 0: RAMP_STATE_STOWED
- 1: RAMP_STATE_DEPLOYED
- 2: RAMP_STATE_MOVING
- 3: RAMP_STATE_FAULT

flag 의미

- pinch_detected = true: 객체 감지됨
- fault_present = true: fault 존재
- alive_toggle: status를 publish할 때마다 토글되는 비트

참고로 현재 레퍼 API의 alive 판정은 alive_toggle 변화 자체를 해석하는 방식이 아니라, 마지막 수신 시점과 timeout 비교로 수행된다.

6. 운전석 ECU 개발자가 쓰는 방법

운전석 ECU는 `DriverECUCAN` 하나를 사용하면 된다.

6.1 역할

- door ECU로 command 전송
- door ECU가 보낸 status polling
- 마지막 status snapshot 처리
- timeout 기반 alive 판정

6.2 기본 사용 순서

1) 객체와 설정 초기화

```
DriverECUCAN driver;  
DriverECUCANConfig cfg;  
  
driverECUCANInit(&driver);  
driverECUCANInitConfig(&cfg);
```

기본값은 다음과 같다.

```
cfg.port_name = "can0";  
cfg.nominal_bitrate = 5000000;
```

필요하면 수정:

```
cfg.port_name = "can0";  
cfg.nominal_bitrate = 5000000;
```

2) open

```
CANStatus st = driverECUCANOpen(&driver, &cfg);
```

이 단계에서 내부적으로 command/status 용 CAN 소켓이 열리고, `port_name`에 맞는 platform binding이 연결된다.

3) command 송신

```
st = driverECUCANSendCommand(  
    &driver,  
    DOOR_CMD_OPEN,  
    RAMP_CMD_NOP,  
    false,  
    false,  
    false  
);
```

입자 의미:

- door_cmd
- ramp_cmd
- ramp_manual
- emergency_stop
- reset_fault

이 함수는 내부에서 command payload를 pack해서 `Exci00`으로 송신한다.

4) status polling

```
bool updated = false;  
uint32 t_now_ms = AppGetTickMs();
```

```
st = driverECUCANPollStatus(&driver, now_ms, &updated);
```

동작은 다음과 같다.

- 새 status frame이 없으면: `updated = false`, 반환값은 `CAN_STATUS_OK`
- 새 status frame이 있으면: 내부 snapshot 갱신 후 `updated = true`

즉, 이 API는 **event callback** 방식이 아니라 **polling** 방식이다.

5) 최신 status 읽기

```
const DriverECUCANStatusSnapshot* s = driverECUCANGetStatus(&driver);
if (s != 0 && s->status_received)
{
    /* s->door_state, s->ramp_state, s->pinch_detected 등 사용 */
}
```

6) alive 확인

```
bool alive = driverECUCANIsStatusAlive(&driver, now_ms, 1000U);
```

- 최근 1000ms 내 status 수신이면 `true`
- 아니면 `false`

7) close

```
driverECUCANClose(&driver);
```

7. door ECU 개발자가 쓰는 방법

door ECU는 `DoorECUCAN` 하나를 사용할 것임.

7.1 역할

- 운전석 ECU가 보낸 command polling
- 마지막 command snapshot 재시
- timeout 시 안전 fallback 작동
- 현재 상태를 status frame으로 송신

7.2 기본 사용 순서

1) 객체와 설정 초기화

```
DoorECUCAN door;
DoorECUCANConfig cfg;

doorECUCANInit(&door);
doorECUCANInitConfig(&cfg);
```

기본값:

```
cfg.port_name = "can0";
cfg.nominal_bitrate = 5000000U;
```

2) open

```
CANStatus st = doorECUCANOpen(&door, &cfg);
```

이 단계에서 내부적으로 `0x100` command frame 수신용 filter가 설정된다.

3) command polling

```
bool updated = false;
uint32_t now_ms = AppGetTicks();

st = doorECUCANPollCommand(&door, now_ms, &updated);
```

동작:

- 새 command가 없으면: `updated = false`, 반환값은 `CAN_STATUS_OK`
- 새 command가 있으면: 내부 snapshot 갱신 후 `updated = true`

4) 최신 command 읽기

```
const DoorECUCANCommandSnapshot* cmd = doorECUCANGetCommand(&door);
if (cmd != 0 && cmd->command_received)
{
    /* cmd->door_command, cmd->ramp_command, cmd->ramp_manual 등 사용 */
}
```

5) command alive 확인

```
bool alive = doorECUCANIsCommandAlive(&door, now_ms, 10000);
```

6) timeout 시 안전 fallback 적용

```
doorECUCANForceSafeCommand(&door);
```

이 함수는 현재 캐시된 command를 아래 값으로 덮어쓴다.

- `door_command = DOOR_CMD_STOP`
- `ramp_command = RAMP_CMD_STOP`
- `ramp_manual = false`
- `emergency_stop = false`
- `reset_fault = false`

즉, command timeout이나 통신 이상 시 application이 이 함수를 호출해서 기본 안전 정지 상태로 되돌릴 수 있다.

7) status publish

```
st = doorECUCANPublishStatus(
    &door,
    false,
    DOOR_STATE_OPENED,
    RAMP_STATE_STOWED,
    false
);
```

입자 의미:

- `pinch_detected`
- `door_state`
- `ramp_state`
- `fault_present`

이 함수는 내부에서:

- status cache 갱신
- `alive_toggle` 변경
- status payload pack
- 0x381으로 송신

을 수행한다.

8) 현재 publish cache 확인

```
const DoorECUCANStatusCache* cache = doorECUCANGetStatusCache(&door);
```

9) close

```
doorECUCANClose(&door);
```

8. 권장 루프 예시

8.1 운전석 ECU 예시

```
DriverECUCAN driver;  
DriverECUCANConfig cfg;  
bool updated;  
uint32_t now_ms;  
  
driverECUCANInit(&driver);  
driverECUCANInitConfig(&cfg);  
driverECUCANOpen(&driver, &cfg);  
  
for (;;)   
{  
    now_ms = AppGetTickMs();  
  
    /* 필요 시 command 송신 */  
    driverECUCANSendCommand(  
        &driver,  
        DOOR_CMD_OPEN,  
        RAMP_CMD_NOP,  
        false,  
        false,  
        false  
    );  
  
    /* status 수신 */  
    updated = false;  
    driverECUCANPollStatus(&driver, now_ms, &updated);  
  
    if (updated)  
    {  
        const DriverECUCANStatusSnapshot* s = driverECUCANGetStatus(&driver);  
        (void)s;  
        /* UI 반영, 상태 머신 반영 등 */  
    }  
  
    if (!driverECUCANIsStatusAlive(&driver, now_ms, 1000U))  
    {  
        /* 통신 이상 처리 */  
    }  
}
```

8.2 door ECU 예시

```
DoorECUCAN door;  
DoorECUCANConfig cfg;  
bool updated;  
uint32_t now_ms;  
  
doorECUCANInit(&door);  
doorECUCANInitConfig(&cfg);  
doorECUCANOpen(&door, &cfg);  
  
for (;;)   
{  
    now_ms = AppGetTickMs();  
  
    /* 필요 시 command 수신 */  
    driverECUCANReceiveCommand(  
        &door,  
        DOOR_CMD_OPEN,  
        RAMP_CMD_NOP,  
        false,  
        false,  
        false  
    );  
  
    /* status 송신 */  
    driverECUCANSendStatus(&door, now_ms, true);  
  
    /* 필요 시 command 송신 */  
    driverECUCANSendCommand(  
        &door,  
        DOOR_CMD_OPEN,  
        RAMP_CMD_NOP,  
        false,  
        false,  
        false  
    );  
}
```

```

/* command receive */
updated = false;
doorECUCANPollCommand(&door, now_ms, &updated);

if (updated)
{
    const DoorECUCANCommandSnapshot* cmd = doorECUCANGetCommand(&door);
    (void)cmd;
    /* door/slope 제어 로직 반영 */
}

if (!doorECUCANIsCommandAlive(&door, now_ms, 1000U))
{
    doorECUCANForceSafeCommand(&door);
    /* 안전 상태 반영 */
}

/* 현재 상태 publish */
doorECUCANPublishStatus(
    &door,
    false,
    DOOR_STATE_MOVING,
    RAMP_STATE_STORED,
    false
);
}

```

9. 실사용 시 기억할 점

- 이 레퍼는 **polling** 기반이다.
- `command/status` 모두 현재 1바이트 **payload**만 쓴다.
- 운전석 ECU는 0x100 송신, 0x305 수신이다.
- door ECU는 0x100 수신, 0x305 송신이다.
- timeout 기반 `alive` 관정은 application이 주기적으로 `now_ms`를 넣어 호출해야 한다.
- timeout 이후 실제 하드웨어를 어떻게 안전 정지할지는 application 로직이 결정해야 한다.
- `doorECUCANForceSafeCommand()`는 수신 **snapshot**을 안전값으로 덮어쓰는 **helper**이지, 별도 CAN frame을 지출 전 송하는 함수는 아니다.

10. 빠른 참고

운전석 ECU

1. `driverECUCANInit()`
2. `driverECUCANInitConfig()`
3. `driverECUCANOpen()`
4. 필요 시 `driverECUCANSendCommand()`
5. 주기적으로 `driverECUCANPollStatus()`
6. `driverECUCANGetStatus()` / `driverECUCANIsStatusAlive()` 사용
7. 종료 시 `driverECUCANClose()`

door ECU

1. `doorECUCANInit()`
2. `doorECUCANInitConfig()`
3. `doorECUCANOpen()`
4. 주기적으로 `doorECUCANPollCommand()`
5. `doorECUCANGetCommand()` / `doorECUCANIsCommandAlive()` 사용
6. 필요 시 `doorECUCANForceSafeCommand()`
7. 주기적으로 `doorECUCANPublishStatus()`
8. 종료 시 `doorECUCANClose()`

